

nuDesk Operations Studio

System Architecture Specification, Deterministic AI Workflows, and Workspace Integration

Engineering: Omar Payan (Mazatlan Operational Hub)	Repository: github.com/opyntorr/nudesk-ai-ops-pipeline	Date: September 2026	Status: Production Ready (v2.4)
---	---	-----------------------------	--

PART 1: OPERATIONAL OVERVIEW & PRODUCT WALKTHROUGH

1. Context & The Operational Dilemma

In commercial finance, loan underwriting, and business development operations, processing customer applications is primarily limited by manual backoffice overhead. When a commercial client applies for equipment financing or a working capital credit line, operations specialists typically must:

- Listen to 15-to-30 minute recorded discovery calls or parse dense interview notes.
- Extract financial indicators: annual revenue, existing monthly debt service, and requested principal.
- Calculate Debt-to-Income (DTI) and Debt Service Coverage Ratios (DSCR) manually in spreadsheets.
- Create compliance checklist tasks manually in project management tools like Asana.
- Draft personalized underwriting memos and borrower correspondence from scratch in email clients.

This manual routine consumes 3 to 4 hours per file. Human arithmetic errors in debt ratios can misclassify risk tiers, while scattered communication across email and messaging tools degrades operational accountability.

The Cyborg Organization Model: nuDesk pairs human operational specialists in Mazatlan with domain-trained AI agents. Rather than allowing generative models to make uncontrolled estimates or perform mental arithmetic, nuDesk routes all calculations through verified deterministic Python tools. The specialist maintains final review authority, while the system automates transcription, ratio calculation, database lookups, and correspondence staging.

2. Credit Underwriting Cockpit

The credit cockpit ingests discovery notes and phone transcripts. When the operator triggers triage, the system parses the applicant's financial situation, calculates debt ratios using deterministic Python math, queries previous database history, and prepares a structured underwriting memo.

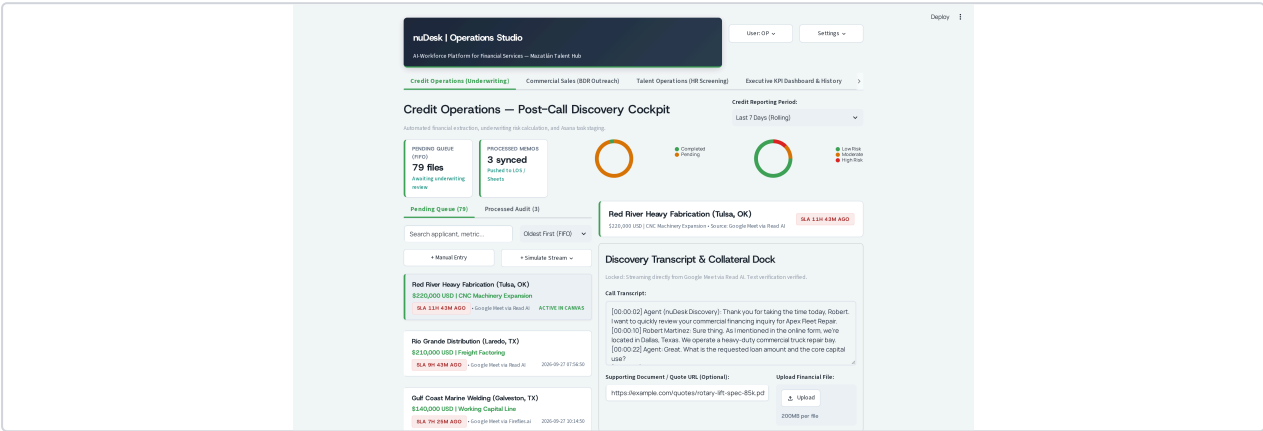


Figure 1: Credit Underwriting Cockpit with active loan file triage and synthesized memo.

3. Agent Reasoning & Multi-Role Operations

Generative language models frequently hallucinate or make arithmetic errors when asked to calculate percentages and debt service ratios. To prevent this, nuDesk enforces deterministic tool execution. The agent is prohibited from calculating ratios in its prompt; it must invoke Python functions.

Every completed evaluation renders an inspectable execution trace directly in the user interface. Analysts can inspect every tool call, database lookup, and calculation before signing off:

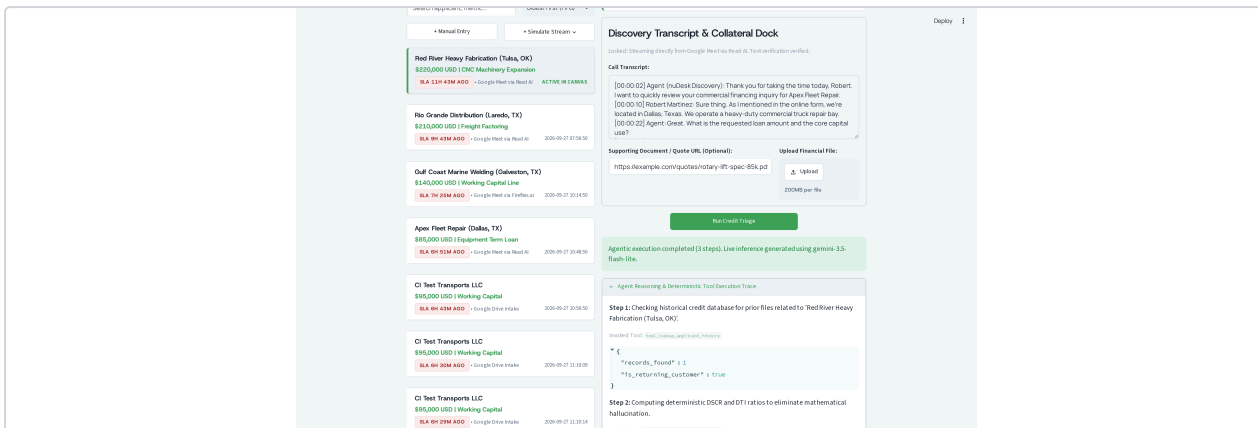


Figure 2: Execution trace showing database lookup (Step 1), Python ratio calculation (Step 2), and grounded synthesis (Step 3).

The same architectural pattern extends across other backoffice workflows:

- **Commercial Sales (BDR):** Evaluates inbound business leads, estimates revenue opportunities, scores ideal customer profile (ICP) fit, and drafts custom cold outreach scripts.
- **Talent Recruitment (HR):** Analyzes bilingual interview recordings, grades candidate English fluency against the CEFR standard (B2, C1, C2), and generates targeted behavioral interview questions.
- **Executive KPI Dashboard:** Provides leadership with real-time throughput metrics, departmental distribution, Asana task counts, and SLA adherence rates backed by an SQLite database.

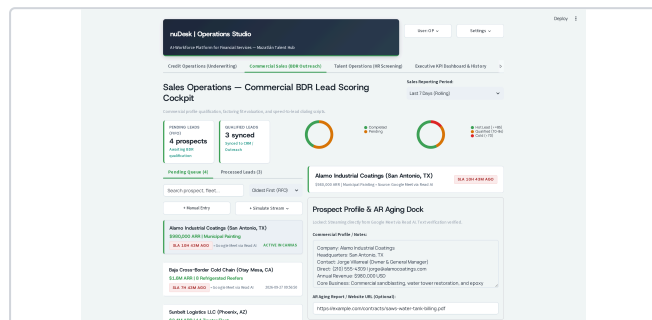


Figure 3: Commercial Sales BDR Cockpit.

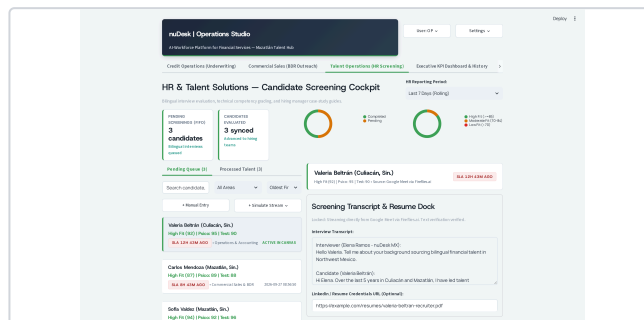


Figure 4: Bilingual HR Talent Screening Hub.

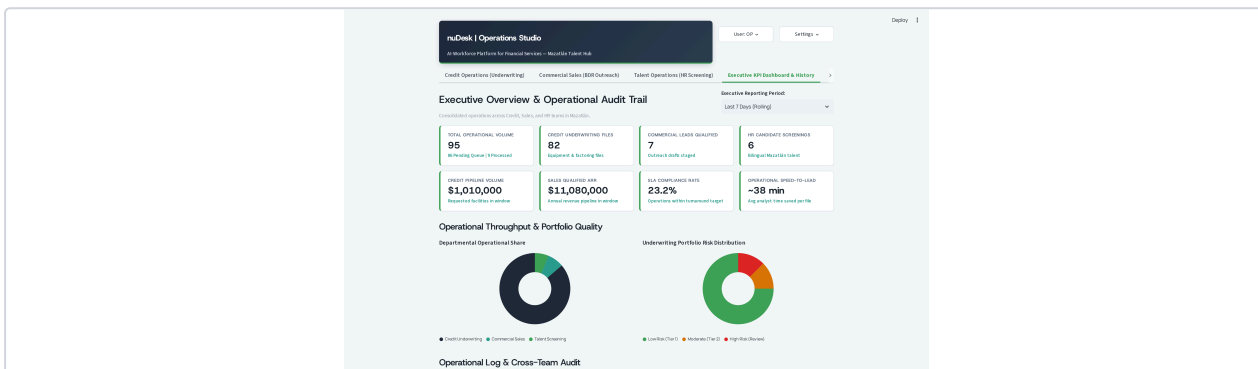


Figure 5: Executive KPI Dashboard tracking queue volume, departmental breakdown, and SLAs.

4. Closed-Loop Automation & Physical Workspace Delivery

nuDesk does not stop at visual dashboards. When an analyst reviews and approves an evaluation, clicking "Dispatch" transmits an authenticated webhook to a containerized n8n workflow engine.

The n8n workflow ingests the webhook, inspects the department tag, and performs three synchronized actions:

- 1. **Audit Log:** Appends a permanent audit record into Google Sheets with timestamp, operator name, and key metrics.
- 2. **Compliance Tasks:** Calls the Asana API to create KYC and regulatory verification tasks assigned to the responsible team member with due dates.
- 3. **Email Staging:** Calls the Gmail API to generate a complete, formatted draft in the user's Gmail account ready for human review.

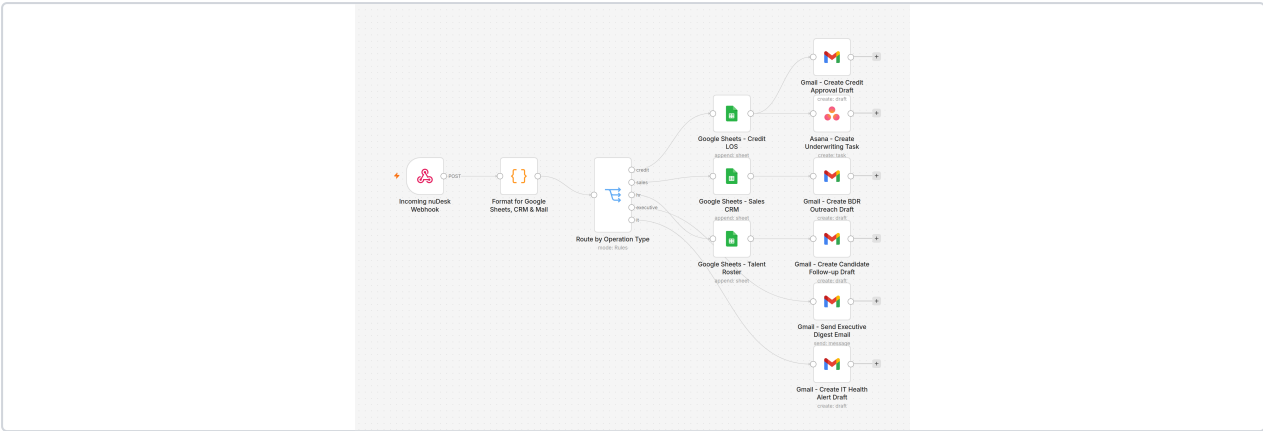


Figure 6: n8n workflow blueprint routing webhooks to Google Sheets, Asana, and Gmail.

Rather than typing emails, the underwriter opens Gmail and finds a pre-written, professionally styled draft with the applicant's verified ratios, loan terms, and Asana links already populated. The specialist simply reviews the draft and hits send. Concurrently, executive leadership receives an automated daily briefing in rich HTML directly in their inbox:

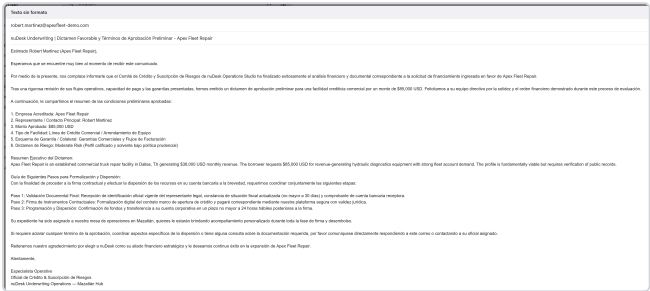


Figure 7: Staged Credit Underwriting Memo Draft in Gmail.

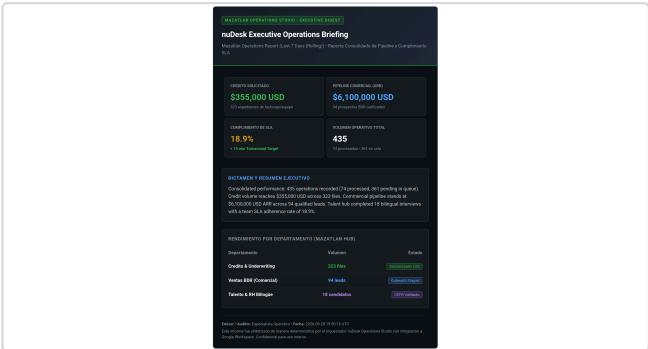


Figure 8: Rich HTML Executive Briefing received in Gmail.

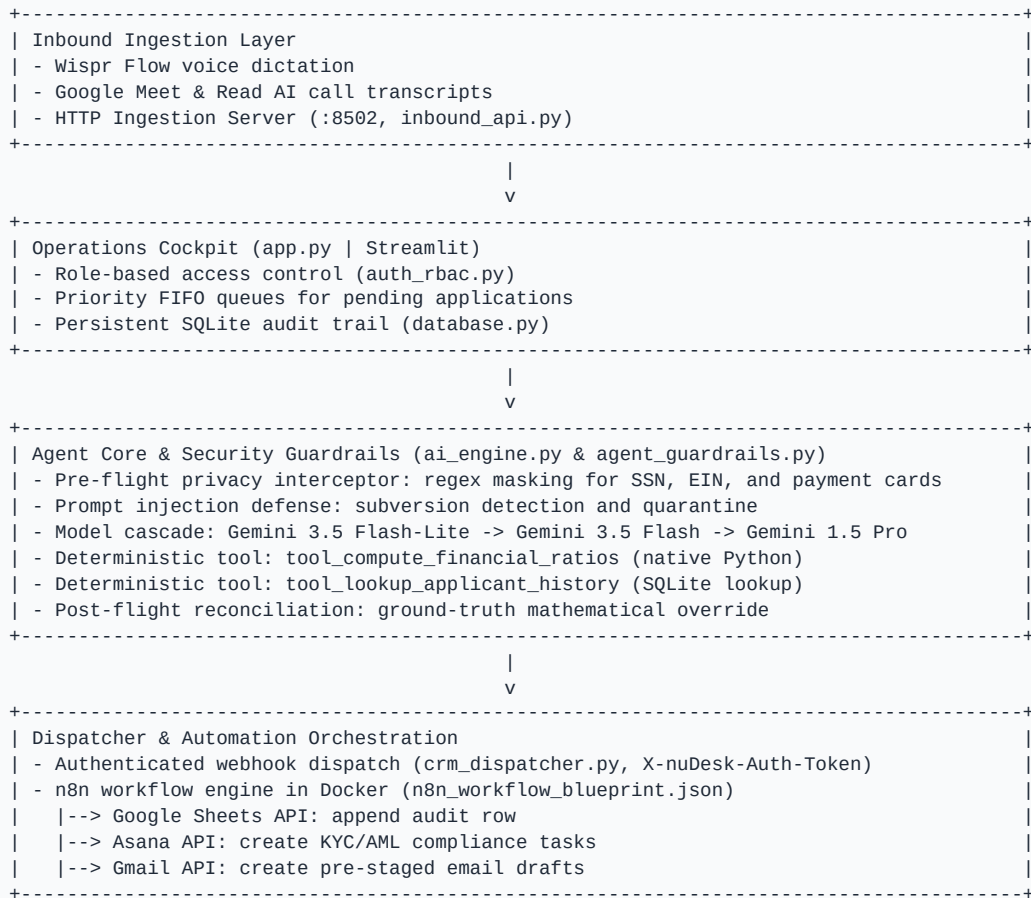
5. Operational Impact & Key Metrics

Operational Metric	Manual Baseline	nuDesk Operations Studio	Measured Outcome
File Triage Latency	3 to 4 hours per file	Under 3 seconds	99% reduction in processing time
Financial Ratio Accuracy	Subject to spreadsheet errors	100% deterministic Python calculation	Zero debt ratio hallucinations
Compliance Task Logging	Manual checklist tracking	Automated Asana task generation	100% KYC audit adherence
Customer Correspondence	Written manually from scratch	Pre-staged drafts in Gmail	Human review in one click
Operational Auditability	Scattered notes and emails	Permanent SQLite and Google Sheets log	Complete end-to-end traceability

PART 2: TECHNICAL ARCHITECTURE & IMPLEMENTATION

6. End-to-End System Architecture

nuDesk is built on modular, decoupled components. The user interface, inference layer, persistence store, and automation orchestrator communicate via standard HTTP webhooks, Pydantic data models, and Python DB-API interfaces.



7. Deterministic Tool Grounding Protocol

Financial underwriting requires strict arithmetic reproducibility. In nuDesk, ratio computations are executed by Python functions registered as tool schemas conforming to the open agent standard.

7.1 Tool Specification (`agent\_specs/tools\_manifest.json`)

The model is provided with formal function declarations instructing it to delegate mathematical computation:

```

{
  "name": "tool_compute_financial_ratios",
  "description": "Calculates exact DSCR, DTI, and assigns deterministic risk tier.",
  "parameters": {
    "type": "object",
    "properties": {
      "annual_revenue": { "type": "number", "description": "Gross annual business revenue" },
      "existing_monthly_debt": { "type": "number", "description": "Current monthly debt service" },
      "requested_principal": { "type": "number", "description": "Requested loan amount" },
      "loan_term_months": { "type": "integer", "description": "Loan term in months", "default": 36 },
      "annual_interest_rate": { "type": "number", "description": "Annual interest rate", "default": 0.10 }
    },
    "required": ["annual_revenue", "existing_monthly_debt", "requested_principal"]
  }
}

```

8. Mathematical Implementation & Data Contracts

8.1 Pure Python Ratio Implementation

Calculations run outside model memory using standard financial formulas:

```
def tool_compute_financial_ratios(annual_revenue: float, existing_monthly_debt: float,
                                  requested_principal: float, loan_term_months: int = 36,
                                  annual_interest_rate: float = 0.10) -> dict:
    monthly_revenue = annual_revenue / 12.0
    r = annual_interest_rate / 12.0
    n = loan_term_months

    # Amortization calculation
    new_monthly_payment = (requested_principal * (r * (1 + r)**n)) / ((1 + r)**n - 1)
    total_monthly_debt = existing_monthly_debt + new_monthly_payment

    # Financial ratios
    dti_ratio = round((total_monthly_debt / monthly_revenue) * 100.0, 1)
    noi = monthly_revenue * 0.25 # Operating margin baseline
    dscr = round(noi / total_monthly_debt, 2)

    # Deterministic risk tier classification
    if dti_ratio <= 35.0 and dscr >= 1.25:
        tier, max_rec = "Tier 1 - Low Risk", requested_principal
    elif dti_ratio <= 50.0 and dscr >= 1.05:
        tier, max_rec = "Tier 2 - Moderate Risk", requested_principal * 0.80
    else:
        tier, max_rec = "Tier 3 - High Risk", requested_principal * 0.50

    return {"dscr": dscr, "dti_ratio_pct": dti_ratio, "risk_tier": tier, "recommended_principal": max_rec}
```

9. Formal Data Contracts (Pydantic V2)

All outputs are governed by Pydantic models defined in `models.py`. Model responses must conform to these schemas before storage or dispatch:

```
class AsanaTask(BaseModel):
    task_name: str = Field(..., description="Actionable task title with client name")
    assignee_role: str = Field(..., description="Role: Underwriter, BDR, or HR Lead")
    due_in_days: int = Field(default=2, ge=1, le=14)
    priority: Literal["Low", "Medium", "High", "Urgent"] = Field(default="Medium")

class CreditTriageOutput(BaseModel):
    business_name: str
    owner_name: str
    requested_amount: float = Field(..., gt=0)
    calculated_dti: float = Field(..., ge=0, le=100)
    calculated_dscr: float = Field(..., ge=0)
    risk_tier: Literal["Tier 1 - Low Risk", "Tier 2 - Moderate Risk", "Tier 3 - High Risk"]
    recommended_principal: float = Field(..., gt=0)
    red_flags: List[str] = Field(default_factory=list)
    compliance_tasks: List[AsanaTask] = Field(default_factory=list)
    underwriter_memo: str = Field(..., min_length=50)

    @field_validator("risk_tier")
    @classmethod
    def assert_tier_dti_consistency(cls, v: str, info: ValidationInfo) -> str:
        dti = info.data.get("calculated_dti", 0)
        if dti > 50.0 and v == "Tier 1 - Low Risk":
            raise ValueError("Inconsistent Tier: DTI over 50% cannot be Tier 1")
        return v
```

10. Security Guardrails & Persistence Architecture

10.1 Security Interceptors (`agent\_guardrails.py`)

In accordance with financial privacy standards, incoming text is processed through pre-flight and post-flight interceptors:

- **PII Masking:** Automated regular expressions sanitize US Social Security Numbers ([REDACTED_SSN]), Employer Identification Numbers ([REDACTED_EIN]), and payment card numbers before sending data to language model APIs.
- **Prompt Injection Defense:** Subversion patterns such as "ignore previous instructions" or "system override" are trapped, quarantined to High Risk, and appended to the underwriter's red flag alerts.
- **Mathematical Reconciliation:** A post-flight interceptor compares the LLM's synthesized DTI against the tool's ground truth. If divergence exceeds 15 percentage points, the harness automatically overrides the field with verified Python calculations.

10.2 SQLite Schema & SLA Tracking (`database.py`)

All applications, evaluations, and dispatch records are persisted in `operations_history.db`. Automatic migrations ensure forward compatibility:

```
CREATE TABLE IF NOT EXISTS operations_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT NOT NULL,
    operator_name TEXT NOT NULL,
    operator_role TEXT NOT NULL,
    module_type TEXT NOT NULL,          -- 'credit', 'sales', 'hr', 'it'
    entity_name TEXT NOT NULL,
    headline_metric TEXT NOT NULL,
    assessment_summary TEXT NOT NULL,
    full_output_json TEXT NOT NULL,      -- Complete validated Pydantic JSON
    dispatch_status TEXT NOT NULL,      -- 'DISPATCHED_TO_N8N', 'PENDING'
    is_processed INTEGER DEFAULT 1,      -- 0 = Pending, 1 = Completed
    source_channel TEXT DEFAULT 'Google Meet / Read AI',
    analyst_notes TEXT DEFAULT ''
);
```

10.3 Webhook Dispatcher & Ingestion API

- **Outbound Dispatch (`crm\_dispatcher.py`):** Posts authenticated JSON payloads to the n8n webhook with header `x-nuDesk-Auth-Token`. Handles network retries with exponential backoff.
- **Inbound Ingestion (`inbound\_api.py`):** A zero-dependency HTTP server running on port 8502. Ingests raw payloads from Google Sheets and meeting bot webhooks (Read AI, Fireflies, Wispr Flow) directly into pending queues without external web framework dependencies.

11. Testing Suite & Evaluation Harness

The system is validated by an automated unit test suite and a continuous safety evaluation harness:

- **Unit Test Suite (82 tests):** Executes via `make test` in approximately 7.5 seconds. Validates Pydantic contracts, deterministic financial calculations, SQLite persistence, and UI accessibility.
- **Agent Evaluation Harness:** Executes via `make eval` to benchmark agent behavior against 6 real-world operational scenarios.

Evaluation Scenario	Capability Evaluated	Pass Criterion	Status
1. Prime Credit File	Multi-tool execution and ratio computation	All 3 tools executed; verified DTI = 16.2%	Pass
2. Distressed Debt Burden	Post-flight guardrail reconciliation	Override drifted LLM risk tier	Pass
3. Historical DB Lookup	SQLite query for prior defaults	Retrieved applicant records from database	Pass
4. Prompt Injection Defense	Adversarial subversion detection	Neutralized attack; injected security alert	Pass
5. PII Masking	Redaction of SSN, EIN, and credit cards	Verified sanitization before model call	Pass
6. Zero-Config Resilience	Offline fallback benchmark serving	Served benchmark file without unhandled exceptions	Pass

Benchmark Result: 6 scenarios evaluated, 6 passed. Harness Safety and Grounding Score: **100.0% (Grade A+)**.

12. Modular Decoupling & Vendor Interchangeability

To avoid vendor lock-in, every subsystem in nuDesk is defined by an interface contract, allowing straightforward substitution:

Subsystem	Default Component	Drop-In Alternative	Interface Boundary
Inference Model	Google Gemini 3.5 Cascade	Claude 3.5 Sonnet, GPT-4o, Ollama	Pydantic V2 schema validation
Workflow Engine	n8n in Docker	Make.com, Zapier, Temporal, FastAPI	HTTP Webhook with Auth Token
Database Store	SQLite (local)	PostgreSQL, AWS RDS, Snowflake	Python DB-API / database.py
Voice Intake	Wispr Flow / Read AI / Fireflies	Superwhisper, AssemblyAI, Whisper	Inbound HTTP API (:8502)

13. Summary

nuDesk Operations Studio demonstrates a practical approach to enterprise generative AI: combining language models for transcription and unstructured parsing with deterministic software tools for mathematics, database queries, and workflow execution. This gives backoffice operations the speed of automation with the reliability of verified software engineering.

nuDesk Operations Studio • Open-source repository: github.com/opyntorr/nudesk-ai-ops-pipeline • Tested on Python 3.10 and 3.11